



Engineering Take-Home Assignment

About Tenex

Tenex is an AI-powered software development agency. Most of us at Tenex would never work at a dev shop and neither should you. We're building something different — a team of the world's best engineers who want to:

- be paid for what they produce
- work on diverse and complex problems
- and work alongside the brightest minds in the world.

Overview

Overview

Congratulations on reaching the take-home assignment. This exercise is meant to test your ability to generate high-quality code, your ability to handle an open-ended request, your product and design taste, your understanding of LLMs and how they work, and your understanding of LLMs, their strengths as they stand today, and their trajectory.

Toggle on the dropdown to review details of the project:

Engineering Project

Choose one of the following projects below and toggle the arrow to review the requirements for each project.

Calendar Assistant

You can build a web or mobile interface for this project. If you decide to build a web interface, use React. If you decide to build a mobile interface, use Expo. Through this interface, a user should be able to authenticate a GSuite account, which should be used to pull calendar information. Display this calendar information in a way that makes sense.

Then, there should be a simple chat interface that allows the user to chat with their calendar agent. The user should be able to say things like “I have three meetings I need to schedule with Joe, Dan, and Sally. I really want to block my mornings off to work out, so can you write me an email draft I can share with each of them?” and “How much of my time am I spending in meetings? How would you recommend I decrease that?”

Talk-to-a-Folder

You can build a web or mobile interface for this project. If you decide to build a web interface, use React. If you decide to build a mobile interface, use Expo. Through this interface, a user should be able to authenticate a GSuite account, which should be used to give google drive access.

Then, a user should be able to copy and paste any link from Google Drive into the interface, which should kick off an agent conversation. This agent should be able to answer any questions about any of the files in the folder. Optimally, citations should be included.

Better-Perplexity

You can build a web or mobile interface for this project. If you decide to build a web interface, use React. If you decide to build a mobile interface, use Expo. Through this interface, a user should be able to communicate with a chat agent with internet search capabilities. This is somewhat trivial, so after completing this, take it one step further in any direction you see fit. This creative leap will be meant to examine your technical strength more than your strengths product / business, so please make sure that it is a technically compelling feature, even if it means it isn't the product decision you would make.

Inbox Concierge

You can build a web or mobile interface for this project. If you decide to build a web interface, use React. If you decide to build a mobile interface, use Expo. Through this interface, a user should be able to authenticate a G-Suite account, which should be used to give gmail access.

On load, group the user's last 200 threads into buckets (Important, Can wait, Auto-archive, Newsletter, etc.) using an LLM-powered classification pipeline you design.

You only need to show the emails with their subject lines and a preview, like the homepage of any email application. Users do not have to be able to click into the emails.

Then, allow the users to create their own buckets, outside of the default options you choose, which should then recategorize all of the emails based on the new buckets.

Smart Recipe Planner

You should build a mobile interface for this project using Expo.

Users should be able to take a photo of ingredients and the app will show them a list of 5 recipes they can make with those ingredients. If they don't like any of those recipes, they can refresh and get 5 new recipes. The recipes should not repeat from refresh to refresh.

Upon finding a recipe they like, the user should be able to click into one of the recipes from the list and get the full recipe in a new screen.

This should not be a chat interface. The recipe list and the recipes themselves should be highly structured.

Submission Instructions

1. **Videos:** unlisted YouTube videos, 10 to 20 min (must be at least 10 min), no scripts and be yourself. Walk us through the following sections:
 - a. Product demo - what you built, why, and the business impact (this should be less

than half the video)

- b. Tech stack - what you picked and why
- c. Architectural decisions - the high-level design and the signals behind it
- d. Technical trade-offs - what you chose not to do, and what you'd do to production-ize or improve the system

2. **Ashby Submission**

- a. Please use the Ashby submission link provided in the initial instructions. If you did not receive a submission link, please follow the instructions below.
 - i. YouTube video links
 - ii. Link to a **public** GitHub repo with a clear [README.md](#) explaining how to run/test the code
 - iii. Deployed/live link